SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I G.Ram Srinivas *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

2. Digital Signature for Document Authentication
A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

3. Preventing Replay Attacks in Online Transactions
An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.
B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

4. Secure Communication over Public Wi-Fi
A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.
B. Recommend a secure communication method and justify how it protects user data.

5. Data Integrity in Cloud Storage
An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.
B. Suggest an additional mechanism to enhance trust and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Answer 1. Secure Password Storage in Web Applications

Scenario: A web application stores user passwords and needs to protect them against database breaches.

A. Why storing plaintext passwords is insecure

Plaintext storage means the actual password is readable by anyone who can read the database. Because the password is the only secret guarding an account, a single leak exposes every user at once.

- **Database breach / SQL injection:** an attacker who dumps the table instantly obtains every credential with no cracking effort.
- **Insider threat:** administrators, developers, and backup operators can see users' passwords.
- **Credential reuse:** users reuse passwords across sites, so one breach leads to credential-stuffing attacks on email, banking and social accounts.
- **Exposure through backups and logs:** plaintext copies spread to backups, logs and replicas that are often less protected.
- **Legal and reputational damage:** violates standards such as OWASP, PCI-DSS, ISO 27001 and data-protection laws, and destroys user trust.
- **Reversible encryption is also weak:** if passwords are merely encrypted, the key must live somewhere on the server; if the key is stolen with the database, all passwords are recovered.

B. Secure method: salted, slow, adaptive password hashing

Recommended method: never store the password. Store only a **salted hash** produced by a deliberately slow password-hashing function such as **Argon2id** (first choice), **bcrypt** or **PBKDF2**.

Procedure:

- **Registration:** generate a unique random salt (16+ bytes) with a CSPRNG; compute *hash = KDF(password, salt, cost)*; store *(salt, cost parameters, hash)*.
- **Login:** read the stored salt, recompute the hash of the entered password and compare with the stored hash using a constant-time comparison.
- **Optional:** add a secret "pepper" held in an HSM or secrets vault, and raise the cost factor over time.

Property	Why it protects passwords
One-way hash	The stored value cannot be reversed to get the password; attackers must guess.
Unique salt per user	Identical passwords give different hashes, which defeats rainbow tables and precomputed lookups and prevents spotting duplicate passwords.
Slow / adaptive cost	Bcrypt, PBKDF2 and Argon2 are intentionally expensive, so brute-force guessing is slowed from billions to thousands of tries per second. Memory-hard Argon2id also resists GPU/ASIC attacks.
Why not plain SHA-256/MD5	General-purpose hashes are extremely fast, so offline cracking with GPUs is cheap.

Justification and conclusion: Even if the database is stolen, the attacker obtains only salted slow hashes, and each password must be cracked individually at great cost. Combined with strong password policies, rate limiting and multi-factor authentication, salted Argon2id/bcrypt hashing is the industry-accepted method and satisfies confidentiality of credentials with minimal risk.

Answer 2. Digital Signature for Document Authentication

Scenario: A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. How digital signatures are used

A digital signature is the public-key equivalent of a handwritten signature. It combines a **hash function** (e.g., SHA-256) with an **asymmetric algorithm** (e.g., RSA or ECDSA). The sender signs with a private key that only the company holds, and anyone can verify with the matching public key, which is distributed through a digital certificate issued by a trusted Certificate Authority (CA).

Signing (at the sender):

- Compute the message digest $h = H(document)$ using SHA-256.
- Encrypt (sign) the digest with the sender's private key: $S = Sign(K_{priv}, h)$.
- Send the document together with the signature S and the sender's certificate.

Verification (at the receiver):

- Validate the certificate against the trusted CA (checks identity, expiry, revocation).
- Decrypt/verify S with the sender's public key to recover the digest h'.
- Independently compute $h = H(received\ document)$. If $h = h'$ the signature is valid; otherwise the document is rejected.

Step	Sender	Receiver
1	Hash the document: h = H(M)	Hash the received document: h = H(M')
2	Sign: S = Sign(private key, h)	Verify: h' = Verify(public key, S)
3	Send M + S + certificate	Accept only if h = h'

B. How integrity and non-repudiation are ensured

- **Integrity:** The signature is bound to the hash of the exact document. Changing even one bit changes the digest (avalanche effect), so the recomputed hash will not match the signed one and tampering is detected. Finding another document with the same hash is computationally infeasible because of collision resistance.
- **Authentication:** Only the holder of the private key can produce a signature that verifies under the public key, and the CA-issued certificate ties that key to the company's identity.
- **Non-repudiation:** The private key is known only to the signer, so the signer cannot later deny having signed the document, and a third party (auditor or court) can verify the signature using the public key. A symmetric MAC cannot provide this because both parties share the same key.
- **Timestamping:** Adding a trusted timestamp proves when the document was signed, even if the certificate is later revoked or expires.

Conclusion: Digital signatures (hash + RSA/ECDSA + PKI certificates) give the company a legally recognised way to guarantee that official documents are genuine, unaltered, and undeniably originate from the company. Protecting the private key (HSM or smart card) is essential for the scheme's security.

Answer 3. Preventing Replay Attacks in Online Transactions

Scenario: An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. How replay attacks occur

In a replay attack the adversary does not need to break the encryption or learn the contents. He simply records a valid, correctly authenticated message and retransmits it later so that the receiver accepts it as fresh.

- **Capture:** The attacker eavesdrops (e.g., on an unsecured network, through man-in-the-middle, or malware) and records a legitimate message such as "Transfer ■5,000 to Account X" along with its valid MAC/signature or token.
- **Resend:** The attacker later sends the identical message to the server.
- **Acceptance:** The server checks the signature or MAC, finds it valid because the message was genuinely created by the user, and processes the payment again. The result is duplicate debits, repeated fund transfers or session hijacking.
- **Root cause:** The protocol provides authenticity and integrity but no *freshness*; the server cannot distinguish a new message from an old copy.

B. Cryptographic countermeasure: nonces, timestamps and sequence numbers with a MAC/signature

Recommended technique: Include a **unique nonce** (random one-time value), a **timestamp** and/or a **sequence number** inside every transaction, and protect the entire message with a **MAC (HMAC-SHA256)** or digital signature, transported over TLS. Challenge–response authentication works the same way: the server issues a fresh random challenge, and the client must answer with a value derived from it.

Mechanism	How it stops replay
Nonce	A unique random number is generated per transaction. The server stores used nonces and rejects any repeat.
Timestamp	Messages carry the time of creation. The server accepts only those within a small window (e.g., 30–60 s), so old captures expire. Needs synchronised clocks.
Sequence number / counter	Each message has an increasing counter; any number already seen or lower than expected is rejected.
HMAC / digital signature over (data + nonce + timestamp)	Binds the freshness values to the payload, so the attacker cannot change the nonce or time to make an old message look new without invalidating the tag.
TLS session keys	Fresh session keys and record sequence numbers per connection prevent replaying records across sessions. One-time tokens (e.g., OTP, 3-D Secure) add transaction-level freshness.

Justification: The attacker can replay the original message, but it will be rejected because its nonce has already been used or its timestamp is stale. If the attacker modifies the nonce or timestamp to make it look new, the HMAC/signature no longer verifies because the attacker does not know the secret key. Thus authenticity, integrity *and* freshness are all enforced, which closes the replay vulnerability with little overhead.

Answer 4. Secure Communication over Public Wi-Fi

Scenario: A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Risks involved

Risk	Explanation
Eavesdropping / packet sniffing	Traffic is sent in clear text over the air; tools such as Wireshark can read passwords, emails, card numbers and session data.
Man-in-the-Middle (MITM)	The attacker positions himself between the user and the access point and can read or alter data in transit.
Evil-twin / rogue hotspot	A fake access point with a legitimate-looking name (e.g., "Airport_Free_WiFi") lures users, giving the attacker full control of their traffic.
Session hijacking	Stealing unencrypted cookies or tokens lets the attacker impersonate the user without the password.
DNS spoofing / SSL stripping	Users are redirected to phishing sites or downgraded from HTTPS to HTTP.
Malware injection and network attacks	Other devices on the same open network can scan the device for vulnerabilities, or inject malicious content/downloads.
Loss of confidentiality, integrity, privacy	Leads to identity theft, financial fraud and corporate data leakage.

B. Recommended secure communication method: VPN with TLS/HTTPS

Recommendation: Connect through a trusted **Virtual Private Network (VPN)** (IPsec or OpenVPN/WireGuard) and access only **HTTPS (TLS 1.2/1.3)** websites and applications, preferably with multi-factor authentication. Where possible, use WPA3 or the mobile data connection instead of open Wi-Fi.

How it protects the user's data:

- **Confidentiality:** The VPN creates an encrypted tunnel (AES-128/256) between the device and the VPN server. Anyone sniffing the Wi-Fi sees only ciphertext.
- **Key exchange:** Session keys are established with Diffie–Hellman/ECDHE and RSA/ECDSA certificates, so no secret key is ever sent over the network, and forward secrecy ensures old sessions stay safe even if long-term keys leak.
- **Authentication:** Certificates and signatures verify the identity of the server, which blocks evil-twin and MITM attacks (a forged certificate triggers a warning).
- **Integrity:** HMAC / AEAD modes (e.g., AES-GCM) detect any modification of packets in transit.
- **Replay protection and privacy:** Sequence numbers prevent replays; the local network operator sees only the encrypted connection to the VPN server, not the sites visited.
- **Defence in depth:** HTTPS with HSTS stops SSL-stripping; keeping the OS updated and the firewall on, and turning off auto-connect and file sharing, reduce residual risk.

Conclusion: A VPN together with TLS/HTTPS converts an untrusted open network into a protected channel, giving the user confidentiality, integrity and authentication of all sensitive communications.

Answer 5. Data Integrity in Cloud Storage

Scenario: An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Using hashing to verify data integrity

A cryptographic hash function (SHA-256 or SHA-3) maps data of any size to a fixed-length digest. It is **deterministic**, **one-way**, shows an **avalanche effect** (a tiny change gives a completely different digest) and is **collision resistant**. These properties make the digest a compact "fingerprint" of the data.

Procedure:

- **Before upload:** the organization computes H_{orig} = *SHA-256(file)* and keeps this digest in a safe place outside the cloud provider's control (local secure store, separate database or blockchain ledger).
- **Upload:** the file is sent to the cloud (over TLS, optionally encrypted).
- **Verification:** when the file is later downloaded or audited, the organization recomputes H_{new} = *SHA-256(file)* and compares it with H_{orig}.
- **Result:** if $H_{new} = H_{orig}$ the data is unaltered; if they differ, the file was corrupted or tampered with (bit rot, insider attack, malware, or a malicious provider).

For large data sets, a **Merkle hash tree** allows individual blocks to be checked efficiently without downloading the whole file.

B. Additional mechanism: digital signature (or HMAC) on the hash

Limitation of plain hashing: a hash alone provides no authentication. An attacker who can modify the file and the stored hash can simply recompute a matching digest, so the hash must itself be protected.

Recommended addition: sign the digest with the organization's private key ($Sig = Sign(K_{priv}, H(file))$), or compute an **HMAC** using a secret key. Verification uses the public key (or the shared secret) and the stored signature, supported by a PKI certificate and, ideally, a trusted timestamp.

Mechanism	Benefit for cloud storage
Digital signature on hash	Provides authenticity, integrity and non-repudiation: proves the data came from the organization and that nobody (including the provider) can forge a valid replacement without the private key.
HMAC	Fast keyed integrity check; only holders of the secret key can generate a valid tag, so attackers cannot recompute it.
Encryption (AES-256-GCM) before upload	Keeps data confidential from the provider, and the GCM authentication tag also detects modification.
Trusted timestamp / audit logs / versioning	Proves when the data existed in a given state and supports forensic review and rollback.

Justification and conclusion: Hashing detects *that* data has changed, and signing or MACing the hash proves *who* produced it and prevents attackers from replacing both data and hash. Together with encryption and secure key management, this layered approach gives the organization strong, verifiable trust in its cloud-stored data without having to fully trust the cloud provider.